

Embedded AI

A practical approach to embedded AI using ESP32

typst.svg

Abdelbacet Mhamdi

2026-07-03

MT @ ISET Bizerte

1. Overview of ESP32 and Embedded AI	2
2. AI	12
3. Basic Circuitry	13

1. Overview of ESP32 and Embedded AI

From REST to WebSocket on ESP32

A progressive web-server architecture: LED → Potentiometer → Video Stream

Foundations: HTTP, REST, WebSocket

HTTP

- Request/response protocol;
- Client asks, server answers, connection closes.

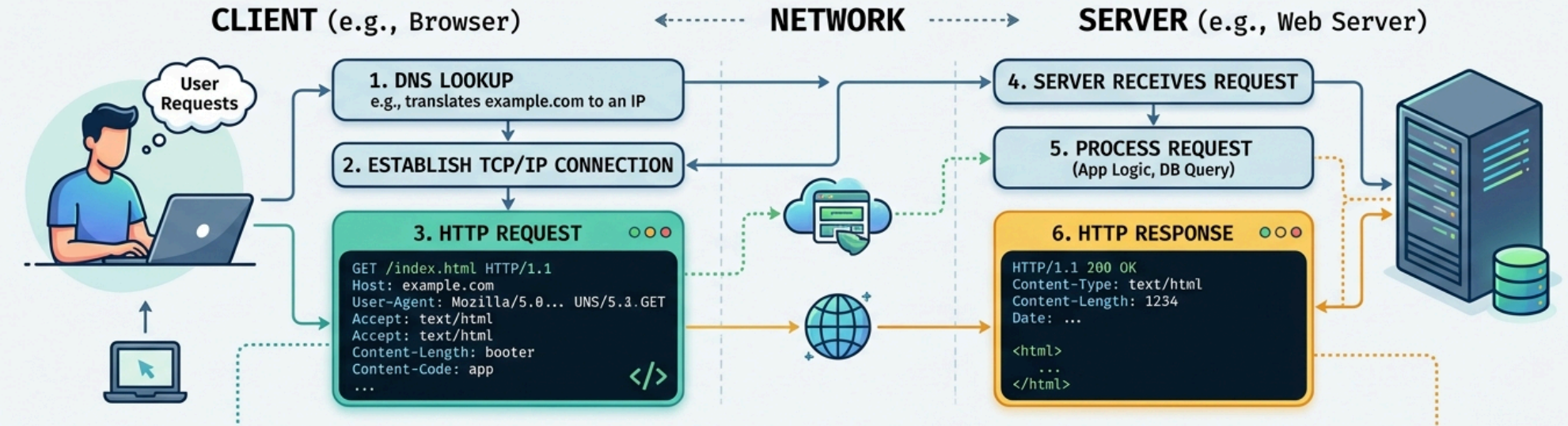
RESTful

An architectural style built on HTTP verbs (GET/POST) and stateless resources.

WebSocket

- A single persistent, full-duplex connection.
- Either side can push anytime.

HTTP (HYPERTEXT TRANSFER PROTOCOL)



HTTP METHODS (Verbs)

GET	GET /index.html HTTP/1.1 Host: example.com Date: text/html	Get rescription and coky makes HTTP methods
POST	GET POST HTTP/1.1 Host: example.com Date: text/html	Descriptions and action method write HTTP methods
PUT	GET PUT HTTP/1.1 Host: example.com Host: ...	Descriptions and merris to the HTTP redirection
DELETE	DELETE POST HTTP/1.1 Host: example.com ...	Brief uncessions and deletets HTTP information

HTTP STATUS CODES

2xx (Success)	200. Success 200. Common us and message
3xx (Redirection)	3xx. Redirection 3xx. Redirection and message
4xx (Client Error)	404. Client Error 404. Common error message
5xx (Server Error)	500. Server Error 500. Server Error message

HTTP HEADERS

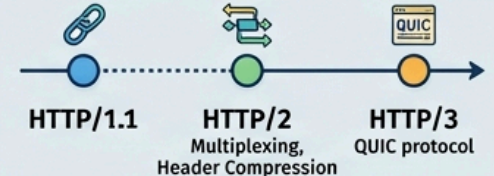
Request

- Request headers: Rescriptionid; packages tests,moung.com, peeroonse
- Response headers: Describionid; content-bss, text/ngime

Response headers

- Response headers: Response, content9px;contenton; response
- Response headers: Response, nonessmons<ton>=flog, context, date-use=https

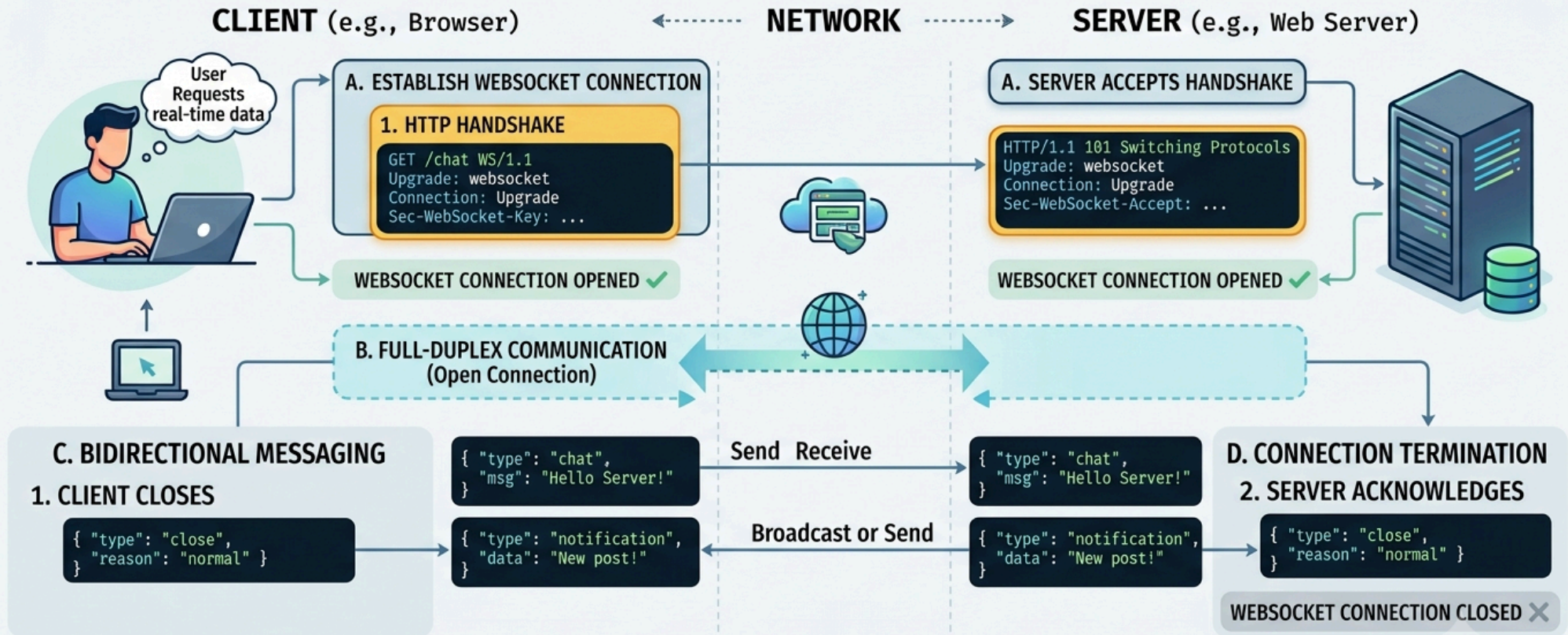
HTTP VERSIONS



HTTPS (HTTP over SSL/TLS)

Added security SSL/TLS to irfs one unvernants and enerwar-ter security

WEBSOCKET (REAL-TIME COMMUNICATION PROTOCOL)



WEBSOCKET CHARACTERISTICS

- Fira Code Font throughout



real-time



persistent



bidirectional



low overhead



client-initiated

NOTE

The three stages hereafter move from stateless HTTP calls toward a persistent, low-latency channel.

1.1.1 Stage 1 — LED Blink (RESTful over HTTP)

HTTP handles this well: turning an LED on/off is an infrequent, stateless action — a plain request/response fits naturally.

RESTful design: the LED is a **resource**.

- GET /led/status → read current state
- POST /led with {"state": "on"} → update state

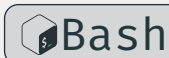
WebSocket is unnecessary here: no continuous data, no need for the server to push updates unprompted.

One request → one response → done.

Toolchain: PlatformIO

A cross-platform build system and library manager for embedded development. It replaces the Arduino IDE's manual library installs and board setup with a declarative `platformio.ini` file.

```
curl -fsSL https://raw.githubusercontent.com/platformio/  
1 platformio-core-installer/master/get-platformio.py -o get-  
platformio.py  
2 python3 get-platformio.py
```



then add `~/platformio/penv/bin` to `PATH`.

INFO

i

One config file, reproducible builds — no more manual Library Manager clicks.

- **Project layout:** `platformio.ini` at the root, source in `src/main.cpp`, dependencies declared under `lib_deps`.
- **Core commands:**
 - Build** `pio run`
 - Flash** `pio run --target upload`
 - Serial output** `pio device monitor`

1.1.2 Stage 2 — Potentiometer (WebSocket)

HTTP becomes inefficient: polling repeatedly for a value that changes continuously wastes bandwidth and adds latency.

RESTful polling would work but doesn't scale to real-time data.

WebSocket solves it: one persistent connection, the ESP32 **pushes** new readings as they change — no repeated handshakes.

Same server, one new /ws endpoint added.

1.1.3 Stage 3 — Video Streaming

- **HTTP** offers a classic trick: `multipart/x-mixed-replace` — a single long-lived response of successive JPEG frames (MJPEG).
- **RESTful** framing doesn't really apply to a continuous stream; this is HTTP used outside the typical request/response mold.
- **WebSocket** is an alternative: binary frames pushed to a canvas, giving more control (e.g. a bidirectional command channel) at the cost of extra client-side decoding work.

Choice depends on whether you need that control channel.

2. AI



3. Basic Circuitry

Digital I/O

Interfacing with the Physical World

Analog I/O

Measuring and Controlling Continuous Signals

Line Encoding Schemes

NRZ-L · NRZ-I · Manchester · Differential Manchester

UART and RS-232

Asynchronous Serial Communication Protocols

Inter-integrated Circuits

A Multi-Master, Multi-Slave, Packet-Switched, Single-Ended, Serial Communication Bus

Serial Peripheral Interface

A High-Speed Synchronous Communication Protocol

Thank you for your attention

Bibliography
